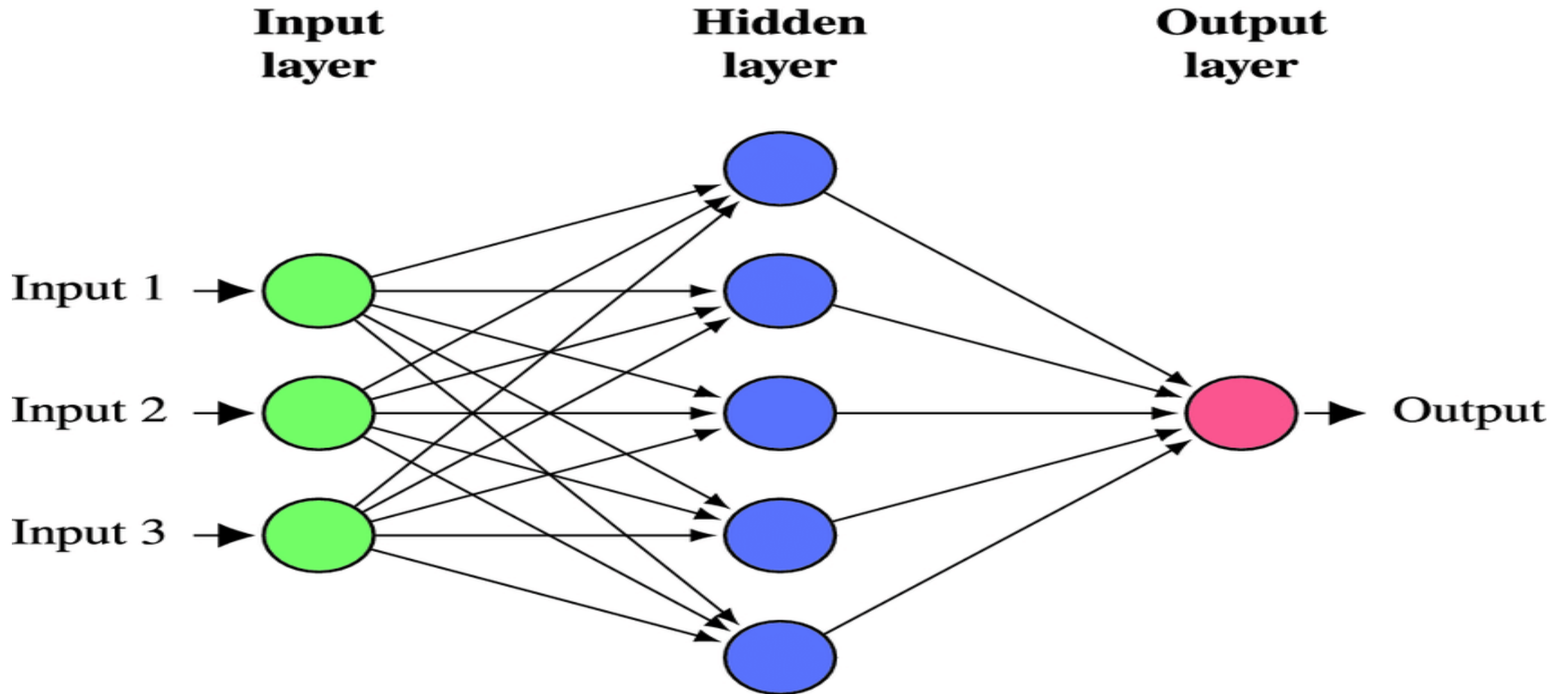


Shallow Neural Network and Convolution Neural Network

Shallow Neural Network:

- A shallow neural network refers to a neural network that consists of only one hidden layer (or very few) between the input and output layers.
- Shallow networks are capable of approximating any function if given sufficient neurons in the hidden layer—a property known as the universal approximation theorem.
- **Applications**
 - Binary Classification Tasks: Email spam vs. non-spam detection
 - Signal Processing: Noise filtering and pattern recognition in signals.



A shallow neural network composed of an input layer of 3 nodes, a hidden layer of 5 nodes, and an output layer of 1 node

Advantage: Shallow Neural Network:

- Simplicity: Speed:
- Less Prone to Overfitting:
- Good for Small Datasets:

Why Do We Go from Fully Connected Layers to Convolutions?

- A fully connected (dense) layer assumes input is a vector:

$$y = Wx + b$$

- Every input neuron connects to every output neuron.

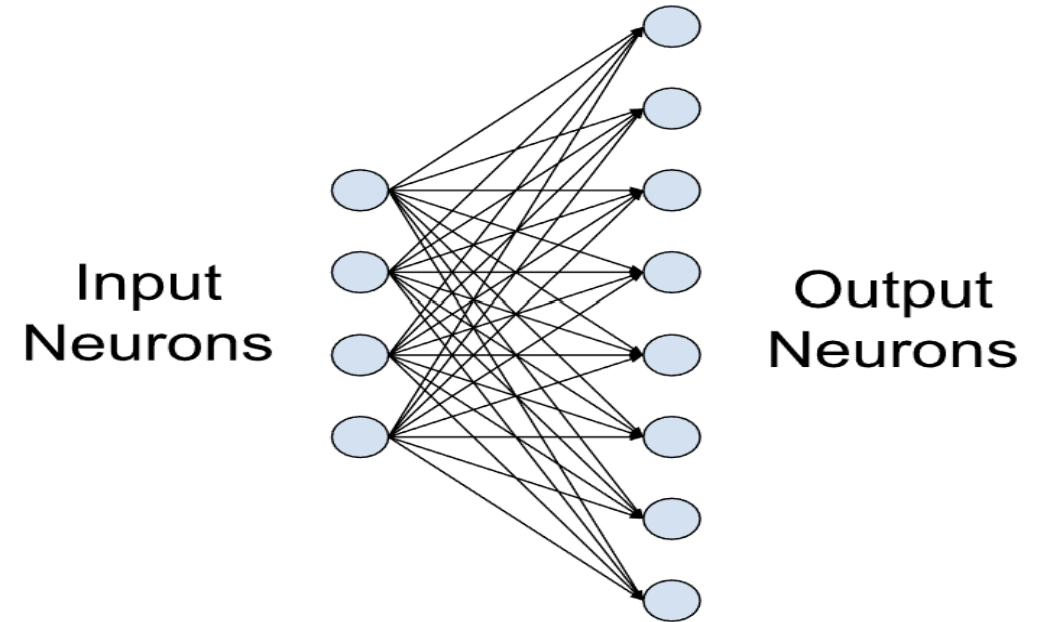
Example:

Input vector: length 100

Output neurons: 50

Total Parameters: $100 * 50 = 5000$

- This is manageable.



Why Do We Go from Fully Connected Layers to Convolutions?

- But Images are not a small vectors

- Example:

- Take a normal color image of Size: 256×256

- Channels: 3 (RGB)

- Total input values: $256 \times 256 \times 3 = 196,608$

- If we connect this to a dense layer of 1000 neurons:

$$\text{Total parameters} = 196,608 \times 1000 = 196,608,000$$

$$\approx 197 \text{ million parameters}$$

Problems:

- Too much memory
- Too slow to train
- Overfits easily
- Not scalable

Dense Layers Ignore Spatial Structure because Dense layers treat every pixel independently

Why Do We Go from Fully Connected Layers to Convolutions?

Given an input image, predict its category

Example: dog, cat, car, airplane

How?

- Images Contain Local Patterns
- Objects are made of smaller visual parts
- A NN learns features **from local** → **global**

Final Layer Objects



Layer 2 Objects



Layer 1 Objects



Convolutions for Images

- Images have a strong spatial structure:
 - nearby pixels are highly related
 - important patterns (edges, corners, textures) appear locally
- Instead of connecting every pixel to every neuron (as in fully connected layers), use convolution operations to detect edges, corners, textures
- Convolution operations serve as feature extractors.
- These operations extract meaningful features like edges, textures, and patterns by applying filters (kernels).
- This process is important for tasks like image classification, object detection etc.

How Convolution Works

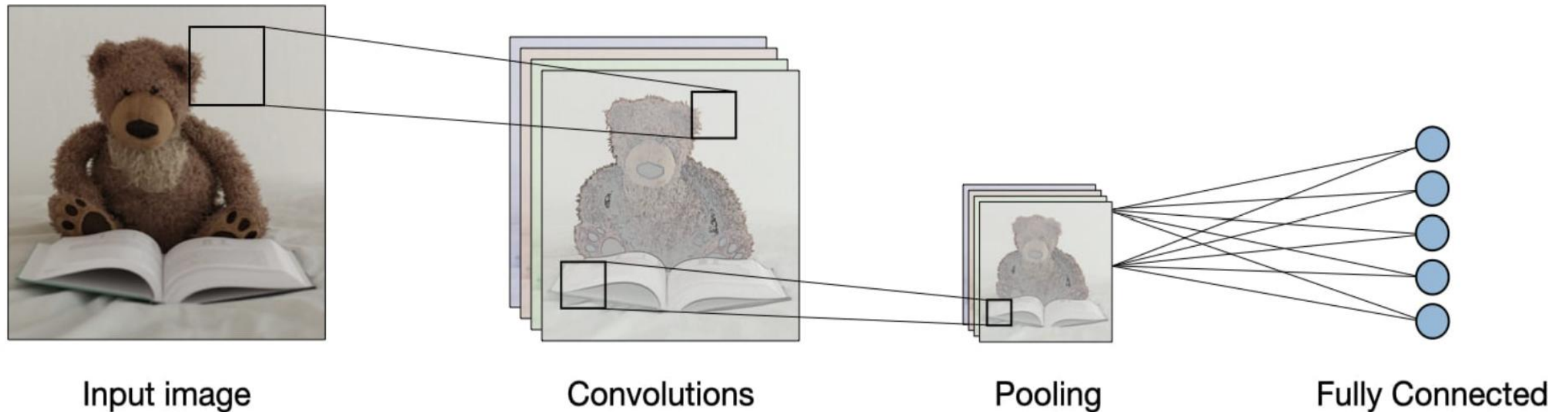
- Convolution involves sliding a kernel (a small matrix) over the input data and performing element-wise multiplication followed by summation.
- The result is a feature map that highlights specific patterns in the input.
- Key parameters include:
 - **Kernel Size:** Determines the area of the input covered by the filter (e.g., 3x3 or 5x5).
 - **Stride:** Specifies the step size for moving the kernel across the input.
 - **Padding:** Adds extra rows/columns to the input to control the output size.

Convolutional Neural Networks

- A CNN is a type of Artificial Neural Network (ANN).
- It is mainly designed to work with grid-like data, such as Images, Videos **OR** It is specifically designed to process pixel and image data
- CNNs extract important features from visual data, like: Edges, Shapes, Objects
- Due to this, CNNs are widely used in computer vision tasks such as:
 - Image classification
 - Face recognition
 - Object detection
- CNNs are very powerful for understanding and analyzing images.

Architecture of CNN

- Convolutional neural networks are generally composed of the following layers:



- Convolution and pooling layer can be fine-tuned with respect to hyperparameters

CNN: Types of layer

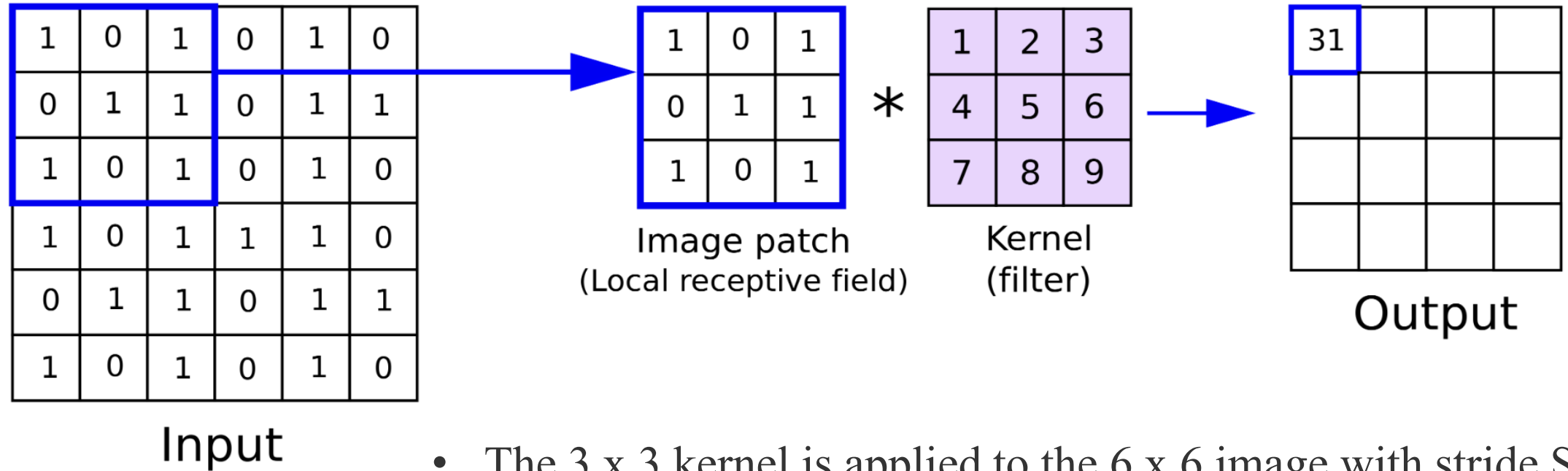
1. Input Layers:

- In this layer, we give input to our model
- In CNN, Generally, the input will be an image or a sequence of images
- This layer holds the raw input of the image
 - Example: An image of dimension $32 \times 32 \times 3$, where width = 32, height = 32 and channel (depth) = 3

2. Convolutional Layers

- This layer is used to extract the feature from the input dataset.
- It applies a set of learnable filters known as the kernels to the input images.
- Filters perform convolution operations
 - Its hyperparameters include the filter size F and stride S . The resulting output O is called feature map or activation map.
 - The filters/kernels are smaller matrices usually 2x2, 3x3, or 5x5 shape.
 - Kernels slides over the input image data I and computes the dot product between kernel weight and the corresponding input image patch
- After convolution, a non-linear activation function like ReLU is applied to introduces non-linearity, so that the network can learn complex patterns.

2. Convolutional Layers



- The 3 x 3 kernel is applied to the 6 x 6 image with stride $S = 1$
- In this case, the output is $1+3+5+6+7+9 = 31$

- Suppose we use a total of 12 filters for this layer, we'll get an output volume (after convolution operations) of dimension 4 x 4 x 12. Total feature map = 12
- After activation function, output volume will have dimensions 4 x 4 x 12.

Why Padding is used?

- Let we have a 5×5 input image and a 3×3 filter and stride $S = 1$

1	2	3	2	1
2	4	6	4	2
3	6	9	6	3
2	4	6	4	2
1	2	3	2	1

Input image

Filter

- The pixel (0, 0) is used only once while the central pixel (2, 2) is used nine times.
- In general, pixels located in the middle are used more often than pixels on edges and corners.
- Therefore, the information on the borders of images is not as it is preserved in the middle

Why Padding is used?

- A simple solution to this problem is padding
- Padding adds rows and columns of zeros to the input image
- If we apply padding P to an input image of size $W \times H$, the output image dimensions become: $(W + 2P) \times (H + 2P)$

- Example: for $P = 2$

3	5	9	1	10
13	2	4	6	11
16	24	9	13	1
7	1	6	8	3
8	4	9	1	9

padding →

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	3	5	9	1	10	0	0
0	0	13	2	4	6	11	0	0
0	0	16	24	9	13	1	0	0
0	0	7	1	6	8	3	0	0
0	0	8	4	9	1	9	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Computing the output dimension of a convolutional layer.

- For an input feature map of size: $n_h \times n_w \times n_c$
- After applying convolution with:
 - Filter size = $f \times f$
 - Stride = s
 - Padding = p
 - Number of filters = n_f

Output Height

$$n_h^{out} = \left\lfloor \frac{n_h - f + 2p}{s} \right\rfloor + 1$$

Output Width

$$n_w^{out} = \left\lfloor \frac{n_w - f + 2p}{s} \right\rfloor + 1$$

Output Depth

$$n_c^{out} = n_f$$

Final output dimension

$$\boxed{n_h^{out} \times n_w^{out} \times n_f}$$

3. Pooling Layers

- Pooling layer reduces the spatial dimensions of the feature maps,
 - makes the computation fast
 - reduces memory and
 - prevents overfitting
- Pooling methods
 - max pooling: Each pooling operation selects the maximum value of the current view
 - average pooling: Each pooling operation averages the values of the current view

The max polling:

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Max Pool
→
Filter - (2 x 2)
Stride - (2, 2)

9	7
8	6

- **4×4 is reduced to 2×2**
- Preserves detected features
- Most commonly used

Stride is the step size by which a filter (kernel) moves across the input image or feature map.

The max polling:

0	1	0
0	0.11	0
0	0.33	0
0	0	0
0	0	0

1	1
0.33	0.33
0.33	0.33
0	0

2 by 2 filter with stride = 1

The average pooling:

2	2	7	3
9	4	6	1
8	5	2	4
3	1	2	6

Average Pool
→
Filter - (2 x 2)
Stride - (2, 2)

4.25	4.25
4.25	3.5

- Downsamples feature map
 - reducing the spatial size (height × width) of that feature map.)
- Used in LeNet

- For an input feature map of size/dimensions: $n_h \times n_w \times n_c$

The dimensions of the output after applying a pooling layer are:

$$\left\lfloor \frac{n_h - f}{s} \right\rfloor + 1 \quad \times \quad \left\lfloor \frac{n_w - f}{s} \right\rfloor + 1 \quad \times \quad n_c$$

where:

- n_h = height of the input feature map
- n_w = width of the input feature map
- n_c = number of channels in the feature map
- f = size of the pooling filter (e.g., 2×2 , 3×3)
- s = stride length

- The resulting feature maps are flattened into a 1-D vector after the convolution and pooling layers so they can be passed into a fully connected layer for categorization or regression.

Why Flattening is Needed?

Because output after convolution and pooling layer: feature maps (3D)

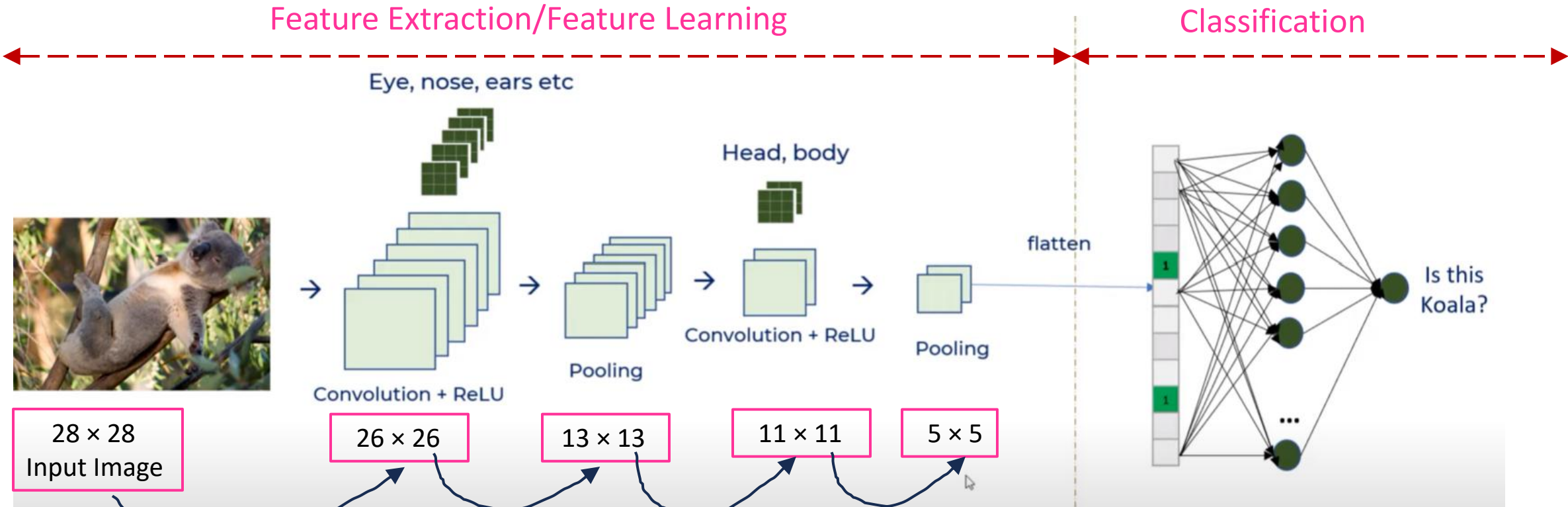
Dense layers require: vectors (1D)

Flattening converts: $H \times W \times C \rightarrow (H \cdot W \cdot C)$

4. **Fully Connected Layers:** The fully connected layer takes the flattened input from the previous layer and learns to make decisions, which are then used by the output layer for final classification or regression
5. **Output Layer:** The output from the fully connected layers is then fed into a logistic function for classification tasks like sigmoid or softmax which converts the output of each class into the probability score of each class.

Convolutional Neural Network (CNN)

Image classification:



- Each pixel stores a value
- For b/w image, channel = 1
- For RGB, channel = 3

$$\text{Output} = [(I - K)/S] + 1$$

Here stride (S) is 1 and number of channel is also 1.
No padding, i.e.; p = 0